



廣東工業大學

《自然语言处理》课程设计

题
地大模型部署与计
用开发

目：基于 Ollama 的本
计算机专业文档问答应

提示一
雷同报告按抄袭处理，判 0 分！
有明显全篇使用 AI 生成痕迹的按抄
袭处理，判 0 分！
提交报告时须删除本提示！

学 院_____

专 业_____

年级班别_____

学 号_____

学生姓名_____

指导教师_____

成 绩_____

摘要

本课程设计面向计算机专业文档问答场景，完成了一个基于 Ollama 本地大模型和检索增强生成（RAG）的问答系统。系统以自建课程文档、公开 CSE 课程数据集样本和 arXiv 计算机论文摘要为知识库，使用 `nomic-embed-text` 生成文本向量，并以 JSON 索引保存文档片段、来源和向量信息。在检索阶段，项目没有直接套用完整向量数据库教程，而是结合向量相似度、`scikit-learn` TF-IDF 关键词分和实体匹配分实现轻量、可解释的混合检索。生成阶段通过本地 Ollama 调用 `qwen2.5:7b-instruct-q4_K_M`，并提供 Streamlit Web 界面和批量评测脚本。实验部分完成了 `full130_20260621` 全量 30 条样本评测，重点比较 `vector` 基线与 `hybrid` 优化策略，检索命中率由 0.7000 提升到 1.0000，参考覆盖度由 0.3669 提升到 0.5404，形成明确的 RAG 优化前后对照。同时补充 Prompt 策略、检索权重和 DeepSeek/Qwen 双模型对比，为课程设计的进阶与提升档要求提供实验依据。

目录

未找到目录项。

第一章 绪论

1.1 任务背景与意义

专业课程资料通常具有概念密集、术语稳定、上下文依赖强的特点。以计算机网络、操作系统、数据库系统和人工智能等课程为例，学生在复习和实验过程中经常会提出“TCP 三次握手为什么需要第三次”“B+ 树为什么适合数据库索引”“RAG 如何降低幻觉”等问题。这类问题既要求模型具备语言组织能力，也要求回答能够回到课程材料和专业文档依据上。仅依赖通用大模型参数记忆，容易出现知识过时、概念混淆和引用依据不清的问题。

检索增强生成 (Retrieval-Augmented Generation, RAG) 把外部知识检索与大模型生成结合起来，适合用于专业文档问答。系统先从知识库中检索与问题相关的片段，再把片段作为上下文交给大模型生成回答。这样既能保留大模型自然语言表达能力，也能让回答更贴近课程资料。本项目选择本地 Ollama 部署，是为了在普通个人电脑上完成可复现的课程设计环境，并避免把课程资料上传到外部云服务。需要强调的是，Ollama 部署在本设计中只是前置条件，主要得分点在任务定义、系统设计、实现细节、30 条以上评测、RAG 优化对照和双模型比较。

1.2 任务定义

本项目对应指导书中的任务 A2：专业文档问答。任务目标是构建一个面向计算机专业资料的本地 RAG 问答系统，使用户能够输入自然语言问题，系统自动检索相关文档片段，并由本地大模型生成带有依据的中文回答。系统的核心不是简单演示模型聊天，而是围绕“专业文档是否被正确检索、参考内容是否覆盖答案要点、生成过程是否可复现”进行设计和评测。

本设计覆盖基础档、进阶层、提升档和加分挑战的主要要求。基础档方面，系统明确了输入输出并提供可运行的 Web 与命令行链路；进阶层方面，使用 30 条评测样本并引入公开数据来源；提升档方面，设计了 vector 纯向量检索基线与 hybrid 三路融合检索优化的严格控制实验；加分挑战方面，增加 deepseek-r1:7b-qwen-distill-q4_K_M 与 qwen2.5:7b-instruct-q4_K_M 的双模型对

比。仓库未实现 LoRA 训练，因此报告不把 LoRA 写作已完成功能。

1.3 输入输出

系统输入包括两类：一类是原始知识库文档，主要为 Markdown、TXT 和可选 PDF 文档；另一类是用户提出的自然语言问题以及评测脚本中的标准问题。知识库文档经过解析、切分、Embedding 和索引构建后保存到 `chroma_db/knowledge_index.json`。虽然目录名保留为 `chroma_db`，但当前实现实际使用 JSON 文件存储索引，并未调用 ChromaDB 客户端。

系统输出包括 Web 页面中的问答结果、命令行脚本输出、检索片段、答案文本以及 `results` 目录下的 CSV/JSON 评测文件。对于单次问答，输出重点包括模型回答、参考片段和耗时；对于批量评测，输出重点包括检索命中率、参考答案覆盖度、推理忠实度、平均检索耗时、平均生成耗时和平均总耗时。

1.4 技术路线概述

项目的技术路线可以概括为“文档采集与清洗—文本切分—Embedding 建库—混合检索—Prompt 构造—Ollama 生成—Web 展示—批量评测”。文档处理阶段由 `src/collect_public_kb.py` 和 `src/build_kb.py` 完成；问答链路由 `src/rag_chain.py` 负责；Web 页面由 `src/app.py` 提供；评测由 `eval.py`、`rag_strategy_compare.py`、`prompt_compare.py`、`model_compare.py`、`weight_tune.py` 和 `full30_runner.py` 完成。

第二章 环境配置与模型选择

2.1 本地运行环境

系统运行在 Windows 本地环境，主要依赖 Python、Ollama、Streamlit、requests、scikit-learn、pypdf 等工具。Ollama 服务通过 Docker 或本地服务暴露到 `http://127.0.0.1:8090`，项目中的 `src/config.py` 已统一配置该地址。报告中不使用 `localhost:11434` 作为当前系统地址，因为该端口不是当前仓库最终配置。

依赖环境通过 `requirements.txt` 管理，核心依赖包括 `requests` 用于调用

Ollama HTTP API, scikit-learn 用于 TF-IDF 特征与关键词分计算, Streamlit 用于构建交互界面, pypdf 用于可选 PDF 文档解析。采用本地环境的主要原因是课程设计需要可复现、可演示和可控制的实验链路, 离线模型和本地索引能降低网络不稳定对实验结果的影响。

2.2 模型配置

系统默认生成模型为 `qwen2.5:7b-instruct-q4_K_M`, 双模型对比中使用 `deepseek-r1:7b-qwen-distill-q4_K_M`, Embedding 模型为 `nomic-embed-text`。`qwen2.5:7b-instruct-q4_K_M` 是指令调优的 7B 量化模型, 在中文问答和简洁输出方面较适合作为默认模型; `deepseek-r1:7b-qwen-distill-q4_K_M` 具有推理模型特征, 适合与 Qwen 在同一 RAG 上下文下比较输出形式和耗时; `nomic-embed-text` 用于把文档片段和问题映射为向量。

选择 7B 量化模型是课程设计中的工程取舍。更大的模型通常可能获得更强语言能力, 但需要更高显存和更长生成时间; 过小模型虽然速度更快, 但专业问答的稳定性不足。7B 量化模型在普通本地设备上更容易部署, 能够支撑 30 条以上样本的完整评测。由于本项目关注 RAG 系统设计和检索优化, 模型部署只是服务能力基础, 不把“能运行 Ollama”单独作为主要贡献。

2.3 参数与端口

当前项目配置中, `OLLAMA_BASE_URL` 为 `http://127.0.0.1:8090`, 默认 Top-K 为 4, 文本切分参数为 `CHUNK_SIZE=420`、`CHUNK_OVERLAP=80`。full30 全量评测使用 `num_predict=64`、`timeout=360`, 并关闭 `context expansion`。这里的“轻量”只体现在生成长度和上下文扩展设置上, 不是减少样本数; `full30_20260621` 仍然是完整 30 条样本全套评测。

第三章 系统设计与实现

3.1 系统总体架构

原始文档/公开数据 → 文档解析与切分 → `nomic-embed-text` 向量化 → JSON 知识库索引 →

vector/hybrid/adaptive 检索 → Prompt 构造 → Ollama 本地模型生成 → Web 展示与批量评测

图 3.1 系统总体架构图

系统整体由知识库构建、检索排序、Prompt 组装、模型生成、前端交互和批量评测六部分组成。知识库构建模块从 data 目录读取自建文档和公开采集文档，生成片段与 Embedding；检索模块根据问题计算向量相似度、TF-IDF 关键词得分和实体匹配得分；生成模块把问题与 Top-K 片段组织成提示词，调用 Ollama 生成回答；评测模块使用统一问题集记录检索命中、参考覆盖和耗时。

3.2 文档解析与文本切分

自建课程文档包括 data/computer_networks.md、data/operating_systems.md、data/database_systems.md 和 data/ai_rag_notes.md，内容覆盖计算机网络、操作系统、数据库系统和人工智能/RAG 基础。公开来源包括 Hugging Face 数据集 hatakekksheeshh/CSE_course_RAG (MIT License)、CCRss/arXiv_dataset (CC0-1.0 metadata) 以及 arXiv Computer Science 论文摘要。src/collect_public_kb.py 默认采用轻量采集方式，不下载 6GB 以上全量数据，而是抽取可追溯的小样本并保存为 Markdown。

公开知识库的采集结果记录在 data/public_kb/manifest.json 中。该 manifest 是一个 dict，其中 documents 字段保存每份公开文档的 title、source、license、url、output_path 等信息，不是顶层 list，也不是空文件。知识库构建阶段会统一读取自建文档和公开文档，并按 CHUNK_SIZE=420、CHUNK_OVERLAP=80 切分。该参数使片段长度足以容纳一个完整概念，又通过 80 字重叠减少边界截断对检索的影响。

3.3 Embedding 与知识库构建

src/build_kb.py 负责文档解析、切分、Embedding 调用和索引写入。系统通过 Ollama 调用 nomic-embed-text，把每个文档片段转换为向量，同时保留片段 id、source、page、char_count 等元数据。当前 chroma_db/knowledge_index.json 中 document_count=41，chunk_count=163，embedding_model=nomic-embed-text。

本项目保留 `chroma_db` 目录名，是为了与早期课程实验习惯兼容，但实际并没有使用 ChromaDB 客户端。相比完整向量数据库，JSON 索引的优点是结构透明、便于查看和调试，适合课程设计、中小规模文档和可解释实验；缺点是面对更大规模知识库时检索效率、并发能力和持久化管理不如 ChromaDB、FAISS 或 Milvus。这个取舍使项目更容易展示 RAG 的关键环节，而不是把复杂度隐藏在数据库框架内部。

3.4 检索策略

系统实现了 `vector`、`hybrid` 和 `adaptive` 三种检索模式。`vector` 是纯向量相似度基线，只根据问题向量与片段向量的相似度排序；`hybrid` 是当前主要优化方案，采用三路融合：`EMBEDDING_SCORE_WEIGHT_V3=0.40`，`LEXICAL_SCORE_WEIGHT_V3=0.45`，`ENTITY_SCORE_WEIGHT=0.15`；`adaptive` 则在 `hybrid` 基础上增加充分性判断和附加检索。

需要区分的是，项目早期曾存在 $0.45 \times \text{向量分} + 0.55 \times \text{TF-IDF 分}$ 的二路融合版本，但 `full30_20260621` 当前结果对应的是三路融合实现。中文计算机专业问答中，TCP、ACID、MVCC、B+ 树、Transformer 等术语具有较强字面信号，纯向量检索有时会把语义相近但术语不完全匹配的片段排在前面；`hybrid` 引入 TF-IDF 和实体匹配后，能更稳定地把包含关键术语的片段排到 Top-K。

3.5 Prompt 构造与 Ollama 生成

`src/rag_chain.py` 根据检索结果构造 Prompt，并调用 Ollama 生成答案。Prompt 策略包括 `baseline`、`stepwise` 和 `reflective`。`baseline` 强调直接依据参考资料回答；`stepwise` 要求按步骤分析，便于观察推理忠实度；`reflective` 要求生成前后进行自查，以减少遗漏。`full30` 评测中统一使用 `qwen2.5:7b-instruct-q4_K_M`、`Top-K=4`、`num_predict=64` 和 `timeout=360`，以控制变量。

生成调用采用 Ollama HTTP API，而不是在线云端模型。这样设计的优点是环境可控、隐私性更好、便于课程答辩现场演示；不足是本地 7B 模型生成速度明显慢于云端高性能推理服务，`full30` 单模型平均生成耗时达到 147.4923s。由于课程设计强调可复现和本地部署，本项目接受这一速度代价，并在评测中明确记录耗时。

3.6 Web 界面与批量评测脚本

src/app.py 使用 Streamlit 构建 Web 页面，用户可以输入问题、选择模型或参数，并查看回答与参考片段。页面入口依赖本地 Ollama 服务 <http://127.0.0.1:8090>。除了交互页面，项目更重视可复现实验脚本：src/eval.py 用于单模型评测，src/rag_strategy_compare.py 用于检索策略对比，src/prompt_compare.py 用于 Prompt 对比，src/model_compare.py 用于双模型对比，src/weight_tune.py 用于权重调优，src/full30_runner.py 用于串联完整 30 条全套增量评测。

第四章 实验与评测

4.1 测试数据与知识库来源

实验问题集来自 eval_questions.csv，共 30 条计算机专业问答样本，覆盖计算机网络、操作系统、数据库、人工智能和 RAG 基础知识。知识库由自建课程文档与公开来源组成，公开来源优先作为补充原始知识库，不用于 LoRA 训练。公开数据采用轻量采样和摘要转 Markdown 的方式，保证仓库可提交、可复现，同时记录来源、许可和 URL。

full30_20260621 是本报告使用的主实验结果。它不是 goal_20260621 的轻量冒烟测试，而是完整 30 条样本全套评测。goal_20260621 仅可作为链路验证参考，不能作为主要质量结论。full30 中的轻量设置是 num_predict=64 和关闭 context expansion，目的是控制本地模型生成时间，不是减少样本数量。

4.2 评测指标

评测指标包括检索命中率、参考答案覆盖度、推理忠实度、平均检索耗时、平均生成耗时和平均总耗时。检索命中率反映 Top-K 片段是否命中预期来源或关键材料；参考覆盖度用自动字符/词覆盖方式近似衡量回答是否覆盖参考答案要点；推理忠实度用于观察回答是否更依赖检索资料而非自由发挥；耗时指标用于比较不同策略的工程代价。

这些自动指标有明确局限。参考覆盖度不是语义等价判断，不能完全代表人工评分；

推理忠实度也会受输出格式影响。因此实验结论主要看趋势和控制变量下的相对变化，尤其是 vector 与 hybrid 的前后对照，而不是把某一个自动分数绝对化。

4.3 单模型效果评测

单模型评测使用 qwen2.5:7b-instruct-q4_K_M，Top-K=4，Prompt 为 baseline，样本数 30，num_predict=64，timeout=360，context expansion 关闭。结果显示检索命中率达到 1.0000，平均参考覆盖度为 0.5585，说明在当前知识库和问题集上，hybrid 检索能够稳定提供相关上下文。平均检索耗时为 0.7879s，而平均生成耗时为 147.4923s，说明系统主要瓶颈来自本地 7B 模型生成。

表 4.1 单模型 full30 评测结果

样本数	模型	Top-K	命中率	参考覆盖度	检索耗时	生成耗时	总耗时
30	qwen2.5:7b-instruct-q4_K_M	4	1.0000	0.5585	0.7879s	147.4923s	148.2807s

4.4 RAG 策略优化对比

为了满足提升档中“必须有明确优化前后对照”的要求，本项目设置了严格的 RAG 策略控制实验。控制变量包括同一知识库、同一 30 条问题、同一模型 qwen2.5:7b-instruct-q4_K_M、同一 Top-K=4、同一生成参数，只切换检索策略。vector 作为优化前基线，hybrid 作为当前优化方案，adaptive 作为附加方案。

实验结果表明，hybrid 相对 vector 将检索命中率从 0.7000 提升到 1.0000，将参考覆盖度从 0.3669 提升到 0.5404，构成清晰的 RAG 检索策略优化前后对照。adaptive 同样达到 1.0000 命中率，但平均总耗时增加到 286.2434s，且参考覆盖度低于 hybrid，因此在当前课程规模和本地推理环境下，hybrid 的性价比更好。

表 4.2 RAG 策略优化对比结果

策略	样本数	命中率	参考覆盖度	检索耗时	生成耗时	总耗时
vector	30	0.7000	0.3669	0.6956s	122.6932s	123.3889s
hybrid	30	1.0000	0.5404	0.7362s	137.6391s	138.3754s

adaptive	30	1.0000	0.3850	124.5705s	161.5900s	286.2434s
----------	----	--------	--------	-----------	-----------	-----------

4.5 Prompt 对比

Prompt 对比实验比较 baseline、stepwise 和 reflective 三种策略。三者均在 hybrid 检索和同一模型下运行 30 条问题。baseline 的参考覆盖度为 0.5286，stepwise 的推理忠实度达到 0.8000，但参考覆盖度降为 0.3782；reflective 的参考覆盖度最高，为 0.6596，但推理忠实度仅 0.0667，平均总耗时也略高。

该结果说明 Prompt Engineering 对输出风格和指标分布有影响，但本项目的主要提升证据仍然放在 RAG 策略优化上。baseline 更适合作为稳定默认策略，stepwise 更适合观察推理过程，reflective 更可能覆盖更多参考要点，但需要人工复核其表达是否真正更准确。

表 4.3 Prompt 策略对比结果

Prompt	样本数	命中率	参考覆盖度	推理忠实度	总耗时
baseline	30	1.0000	0.5286	0.0000	138.9987s
stepwise	30	1.0000	0.3782	0.8000	134.8031s
reflective	30	1.0000	0.6596	0.0667	142.3079s

4.6 双模型对比

双模型对比使用 deepseek-rl:7b-qwen-distill-q4_K_M 和 qwen2.5:7b-instruct-q4_K_M，在同一知识库、同一 30 条问题、同一 Top-K=4 和同一 Prompt 设置下运行。两种模型成功率与检索命中率均为 1.0000，说明检索链路对模型无关，差异主要体现在生成内容和输出格式。

DeepSeek 的参考覆盖度为 0.0000 不能简单等同于语义完全错误。该现象更可能来自当前自动字符/词覆盖指标与其输出格式或内容组织方式不匹配，例如模型输出包含推理痕迹、表达方式与参考答案字面重合少，导致覆盖指标失效。课程答辩中可展示双模型对比作为加分材料，但更严谨的模型质量比较需要引入人工评分或语义相似度指标。

表 4.4 双模型对比结果

模型	样本数	成功率	命中率	参考覆盖度	总耗时
deepseek-r1:7b-qwen-distill-q4_K_M	30	1.0000	1.0000	0.0000	142.2310s
qwen2.5:7b-instruct-q4_K_M	30	1.0000	1.0000	0.5382	146.9947s

4.7 权重调优

权重调优实验在三路融合检索中比较不同 embedding、lexical 和 entity 权重组合。最优组合为 emb=0.30、lex=0.55、entity=0.15，命中率为 1.0000，平均分差为 0.5419。当前默认 full30 实现使用 emb=0.40、lex=0.45、entity=0.15，也能达到 1.0000 命中率，但分差低于最优组合。

权重调优说明关键词项在当前中文计算机专业问答集上对排序有明显帮助。专业术语的字面匹配能够弥补纯向量相似度在短问题、缩写词和概念边界上的不稳定。未来如果知识库规模扩大，可以在 hybrid 之后加入 reranker，把粗召回与精排序分开。

表 4.5 检索权重调优结果

emb	lex	entity	命中率	平均分差
0.30	0.55	0.15	1.0000	0.5419
0.35	0.50	0.15	1.0000	0.5026
0.40	0.45	0.15	1.0000	0.4626
0.45	0.40	0.15	1.0000	0.4191
0.50	0.35	0.15	1.0000	0.3747

4.8 结果分析与不足

从 full30 结果看，项目已经满足 30 条以上样本评测和优化前后对照要求。hybrid 在命中率和参考覆盖度上优于 vector，说明检索策略优化是有效的；Prompt 对比和双模型对比为答辩展示提供了更多维度；权重调优则进一步说明系统不是固定参数演示，而是可以围绕指标进行实验分析。

不足也比较明确。第一，自动参考覆盖度只是近似指标，不能完全代表语义质量，

DeepSeek 覆盖度为 0.0000 的现象尤其需要人工复核。第二，本地 7B 模型生成耗时较长，完整评测成本高。第三，JSON 索引不适合超大规模知识库，在并发检索和增量更新方面能力有限。第四，公开知识库默认采用轻量抽样，没有下载全量 CSE 数据和论文 PDF，因此覆盖面仍然有限。

第五章 总结与展望

本课程设计完成了一个面向计算机专业文档问答的本地 RAG 系统。系统从文档采集、文本切分、Embedding 建库、检索排序、Prompt 构造、Ollama 生成、Web 交互到批量评测形成完整闭环。与简单模型聊天或照搬教程不同，本项目使用 JSON 索引保存知识库，结合 scikit-learn TF-IDF 与向量相似度实现轻量可解释检索，并通过 full130_20260621 的 30 条样本实验给出结果分析。

项目最核心的提升证据是 RAG 策略优化对照：在控制模型、问题集、知识库和 Top-K 的条件下，hybrid 相对 vector 把检索命中率从 0.7000 提升到 1.0000，把参考覆盖度从 0.3669 提升到 0.5404。该实验直接回应了评分细则中“Prompt Engineering / 模型组合 / RAG 优化之一必须有明确优化前后对比”的要求。双模型对比、Prompt 对比和权重调优则进一步展示了系统可扩展的实验能力。

后续工作可以从五个方向展开：一是把 JSON 索引迁移到 ChromaDB、FAISS 或 Milvus，提升大规模检索和增量更新能力；二是扩大公开数据集规模，下载更多课程材料和论文 PDF；三是引入 reranker、语义相似度指标和人工评分，提高评测可靠性；四是优化本地推理性能，例如调整量化模型、缓存 Embedding 和缩短 Prompt；五是在有足够训练数据和算力时尝试 LoRA 微调。但本项目当前没有实际完成 LoRA 训练，因此不把它写成既有成果。

参考文献

- [1] DataWhale. 动手学 Ollama / handy-ollama 教程文档.
- [2] Ollama. Ollama Documentation: Models, API and Local Deployment.
- [3] LangChain. LangChain Python Documentation.

- [4] LangChain. langchain-ollama Integration Documentation.
- [5] Streamlit. Streamlit Documentation.
- [6] scikit-learn Developers. scikit-learn User Guide.
- [7] Lewis P., Perez E., Piktus A., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.
- [8] Vaswani A., Shazeer N., Parmar N., et al. Attention Is All You Need. NeurIPS, 2017.
- [9] Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP-IJCNLP, 2019.
- [10] Johnson J., Douze M., Jegou H. Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 2019.
- [11] Gao Y., Xiong Y., Gao X., et al. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv, 2024.
- [12] Asai A., Wu Z., Wang Y., et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR, 2024.
- [13] Sarthi P., Abdullah S., Tuli A., et al. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. ICLR, 2024.
- [14] Guo Z., Xia L., Yu Y., et al. LightRAG: Simple and Fast Retrieval-Augmented Generation. arXiv, 2024.
- [15] Qwen Team. Qwen Technical Report and Model Documentation.
- [16] DeepSeek-AI. DeepSeek-R1 Model Documentation.
- [17] Hugging Face. hatakekksheeshh/CSE_course_RAG Dataset Card.
- [18] Hugging Face. CCRss/arXiv_dataset Dataset Card.
- [19] arXiv. Computer Science Archive.

[20] 自然语言处理课程设计指导书与评分细则.

附录

附录 A 核心代码文件说明

src/config.py: 模型、端口、路径、Top-K、切分参数和检索权重配置。

src/collect_public_kb.py: 采集公开 CSE 数据集样本、arXiv 元数据和论文摘要，生成 manifest。

src/build_kb.py: 解析文档、切分文本、调用 nomic-embed-text、生成 JSON 知识库索引。

src/rag_chain.py: 实现 vector、hybrid、adaptive 检索，构造 Prompt，调用 Ollama，并进行忠实度检查。

src/app.py: Streamlit Web 页面，提供本地交互式问答入口。

src/eval.py: 单模型 30 条问题评测。

src/rag_strategy_compare.py: vector、hybrid、adaptive 检索策略对比。

src/prompt_compare.py: baseline、stepwise、reflective Prompt 对比。

src/model_compare.py: DeepSeek 与 Qwen 双模型对比。

src/weight_tune.py: 三路融合检索权重调优。

src/full30_runner.py: 完整 30 条全套增量评测 runner。

附录 B 主要运行命令

```
curl http://127.0.0.1:8090/api/tags
```

```
python src/collect_public_kb.py
```

```
python src/build_kb.py
```

```
streamlit run src/app.py
```

```
python src/rag_chain.py --question "RAG 的基本流程是什么"
```

```
python src/full30_runner.py --tasks eval --suffix full30_20260621 --top-k 4  
--prompt baseline --num-predict 64 --timeout 360 --resume
```

```
python src/full30_runner.py --tasks strategy --suffix full30_20260621 --top-k  
4 --prompt baseline --num-predict 64 --timeout 360 --resume
```

```
python src/full30_runner.py --tasks prompt --suffix full30_20260621 --top-k 4  
--prompt baseline --num-predict 64 --timeout 360 --resume
```

```
python src/full30_runner.py --tasks model --suffix full30_20260621 --top-k 4  
--prompt baseline --num-predict 64 --timeout 360 --resume
```

```
python src/full30_runner.py --tasks weight --suffix full30_20260621 --top-k 4  
--prompt baseline --num-predict 64 --timeout 360 --resume
```

附录 C 结果文件说明

results/eval_summary_full30_20260621.json: 单模型 30 条评测汇总。

results/eval_results_full30_20260621.csv: 单模型 30 条评测明细。

results/rag_strategy_compare_summary_full30_20260621.json: RAG 策略对比汇总。

results/rag_strategy_compare_results_full30_20260621.csv: RAG 策略对比明细。

results/prompt_compare_summary_full30_20260621.json: Prompt 对比汇总。

results/prompt_compare_results_full30_20260621.csv: Prompt 对比明细。

results/model_compare_summary_full30_20260621.json: 双模型对比汇总。

results/model_compare_results_full30_20260621.csv: 双模型对比明细。

results/weight_tune_results_full30_20260621.json: 检索权重调优结果。

goal_20260621 相关结果仅用于链路验证，不作为本报告主要质量结论。